G.H. Raisoni International Skill Tech University
Department of Mechanical Engineering

Unit 1 :Introduction to Data Structure

Prof. Sagar Mahajan

Data Structures – Definition, Need & Classification



Organize It!



Definition



A **Data Structure** is a specific way of **organizing, storing, and managing data** in a computer so it can be accessed and modified **efficiently**.

The Need

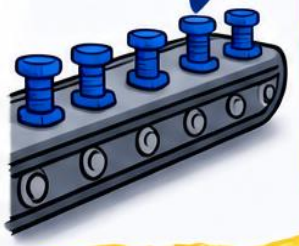


Just like a messy workshop wastes time, **unorganized data** slows down the computer.

Data Structures provide the **"right container"** for the data.

Classification

Like a Single Conveyer Belt



Linear (In a sequence)

Data is arranged one after another.

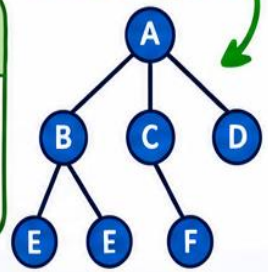
(e.g., Arrays, Stacks, Queues, Linked Lists)

Non-Linear (Hierarchical)

Data is arranged in a hierarchy or network.

(e.g., Trees, Graphs)

Like a Complex Bill of Materials



Speaker Notes: *"Imagine you need an M10 bolt in a messy workshop. It takes you 10 minutes to find it. If you organize your workshop with labeled bins, it takes 10 seconds. That is exactly why we need Data Structures. We classify them into 'Linear' (like a single conveyor belt) and 'Non-Linear' (like a complex Bill of Materials for a car engine)."*

Organize Data... Build Efficiency!

Problem Solving & Introduction to Algorithms

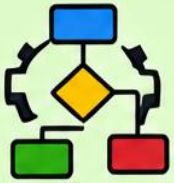
The Big Picture

Problem



"Something needs to be solved!"

Algorithm



"A step-by-step plan!"

Code



"Implement the plan!"

Output



"Get the solution!"

Standard Operating Procedure (SOP)



An Algorithm is just like an SOP – it tells you exactly what to do!



Problem Solving



Problem Solving:

The process of taking a **real-world issue**, defining it mathematically, **step-by-step solution**.

Algorithm

An Algorithm:

A finite set of step-by-step instructions to solve a specific problem.

(It is the **logic**, not the code).

Characteristics of a Good Algorithm

1. **Unambiguous:** Every step must be clear and have only one meaning.
2. **Input/Output:** Must have zero or more inputs and at least one output.
3. **Finiteness:** It must terminate (stop) after a finite number of steps.
4. **Effectiveness:** Every step must be basic enough to be carried out.



"An algorithm is exactly like a **Standard Operating Procedure** (SOP) on a factory floor. It tells you exactly how to machine a part. But a bad SOP will ruin the part. A good algorithm must be **crystal clear**, must **finish eventually** (no infinite loops), and must **give a result.**"



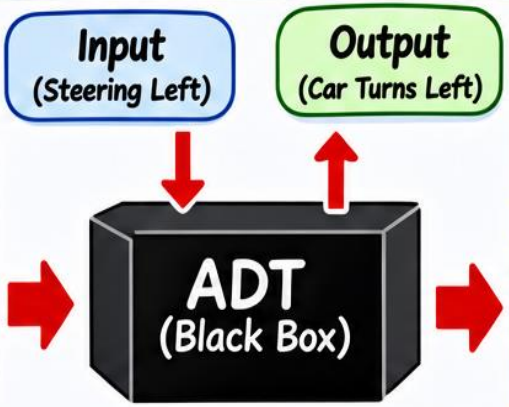
Plan Well → Solve Smart!

Abstract Data Types (ADT)

What is an ADT?

It defines **what** operations can be performed on the data, but completely hides **how** they are implemented.







Two Parts

1 Public Interface

The operations (e.g., "**Add tool**", "**Remove tool**").

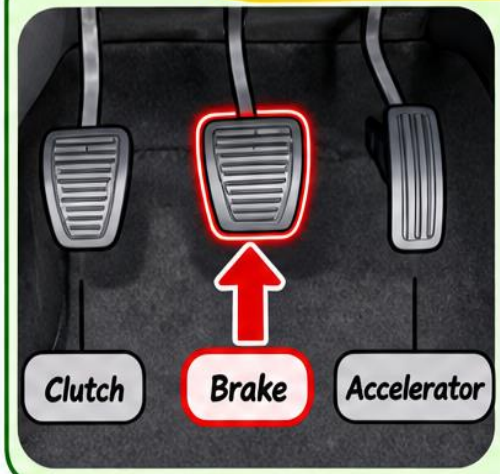


2 Hidden Implementation

The actual code (how the tool is added).



Mechanical Analogy: Car Pedals



As a driver, you know pressing the **brake** stops the car **(Interface)**.

You don't need to know the hydraulic pressure calculations inside the brake lines **(Implementation)**.

ADT gives a **simple set of operations** (like a steering wheel or brake pedal), while **hiding** the complex engineering underneath (like the internal mechanics).

Simple Interface... Complex Power!

Asymptotic Notations (Big O, Omega, Theta)

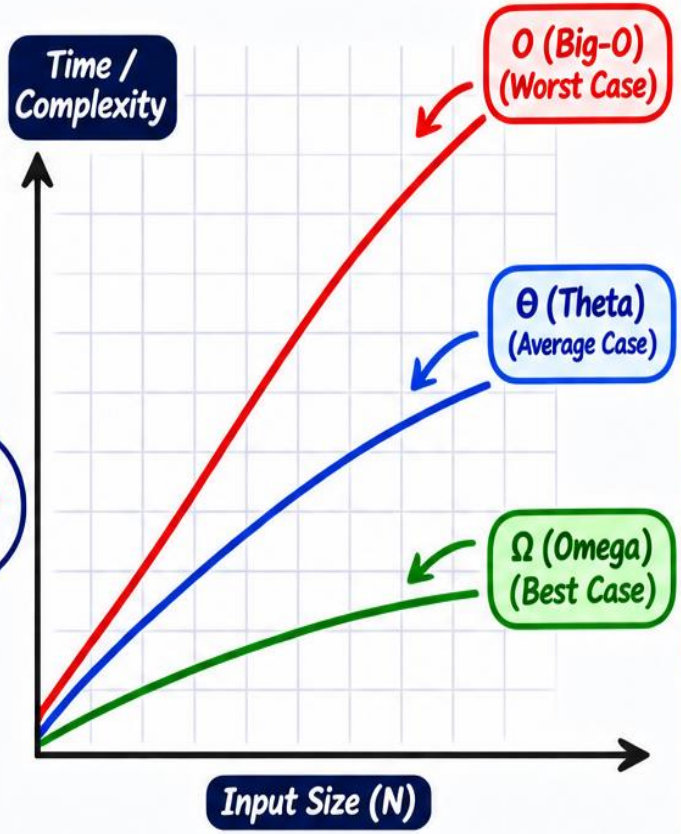
What is Asymptotic Notation?

A mathematical tool to measure the **efficiency (Time/Space)** of an algorithm as the input data **(N)** gets massive.

We don't measure in seconds; we measure in **growth rate**.



The Three Notations & Their Meaning



1. Big-O (O) – The Worst Case



The maximum time an algorithm will ever take. **(Upper Bound)**.

Analogy: The absolute worst traffic jam on your way to work.

2. Omega (Ω) – The Best Case



The minimum time it will take. **(Lower Bound)**.

Analogy: Green lights all the way to work.

3. Theta (Θ) – The Average Case



The realistic, expected time. **(Tight Bound)**.



Big-O is the most important one. It asks:

"If the data doubles, how much slower does it get?"

Big-O is the worst-case guarantee. We always design systems based on the **worst-case scenario**.

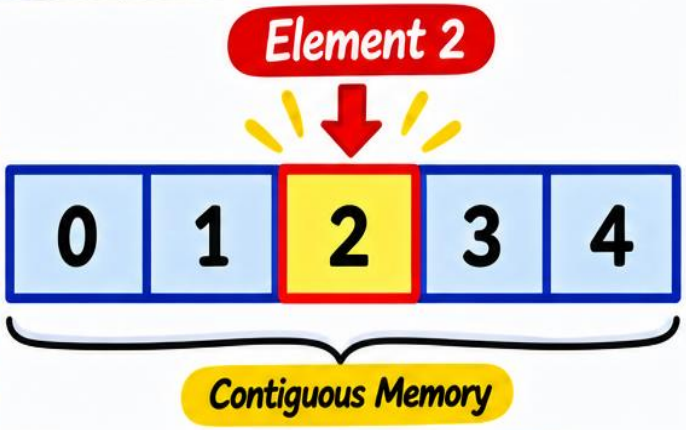
Analyze Growth... Not Time!

Arrays – Representation & Operations

Representation

An Array is a collection of elements of the **same data type** stored in **contiguous (continuous) memory.**

(Element index starts at 0).

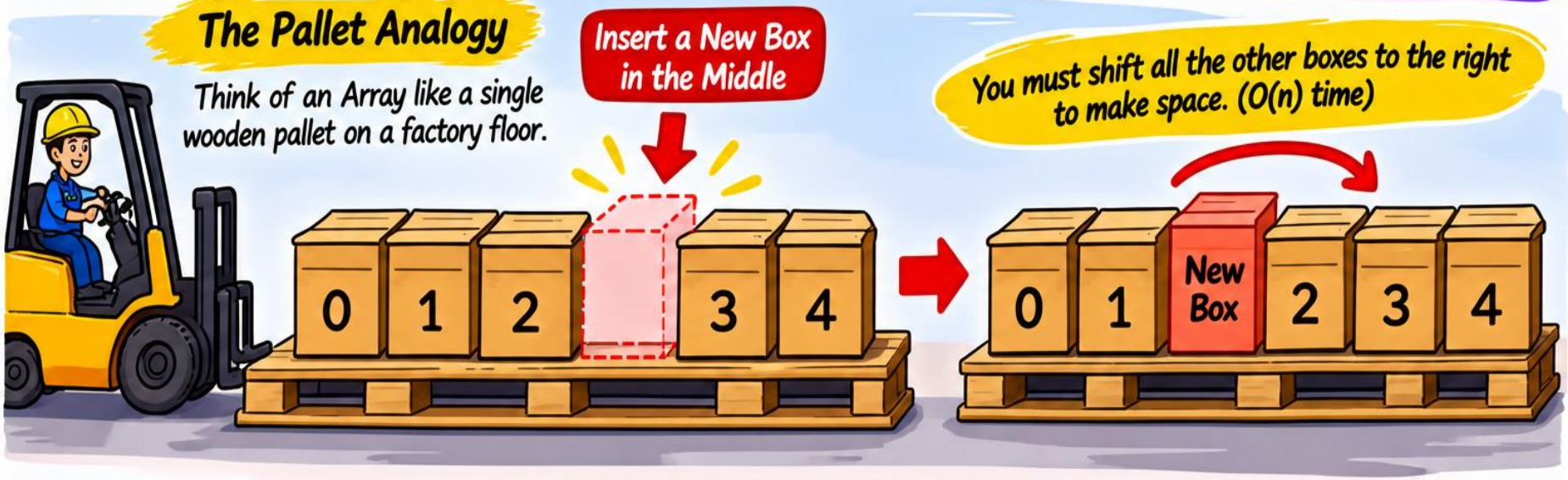


Operations

1. **Traversal:** Visiting every element one by one.
2. **Searching:** Finding an element (Fast if using Binary Search, O(log n)).
3. **Insertion & Deletion:** Putting in or removing an element.

The Pallet Analogy

Think of an Array like a single wooden pallet on a factory floor.



The Big Problem: Arrays have a **fixed size**. If you want to insert a new box in the middle, you must **shift all the other boxes over**. **(O(n) time – slow for massive arrays).**

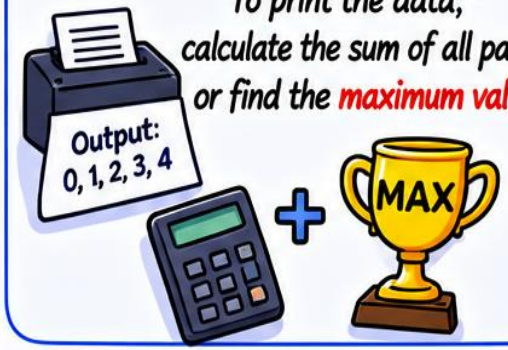
Array Operation 1 – Traversal

What is Traversal?

Visiting every element of the array **exactly once**, from the first index **(0)** to the last index **(N – 1)**.



Why do we do it?



To print the data, calculate the sum of all parts, or find the **maximum value**.

Time Complexity

O(n)

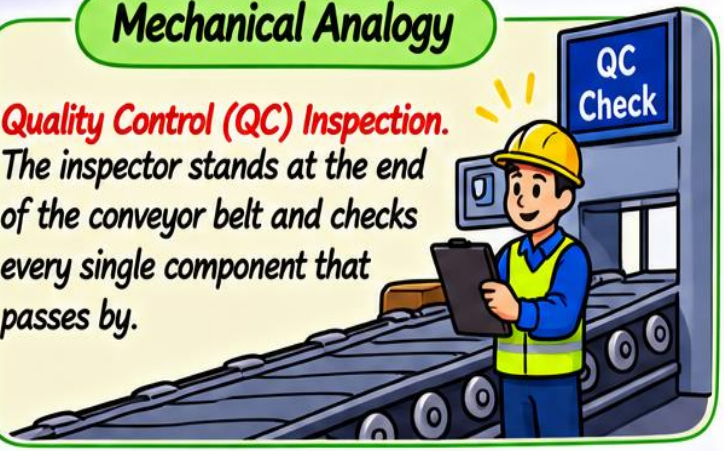
(Linear Time)



If you have **100** parts, it takes **100 steps**.

Mechanical Analogy

Quality Control (QC) Inspection.
The inspector stands at the end of the conveyor belt and checks every single component that passes by.



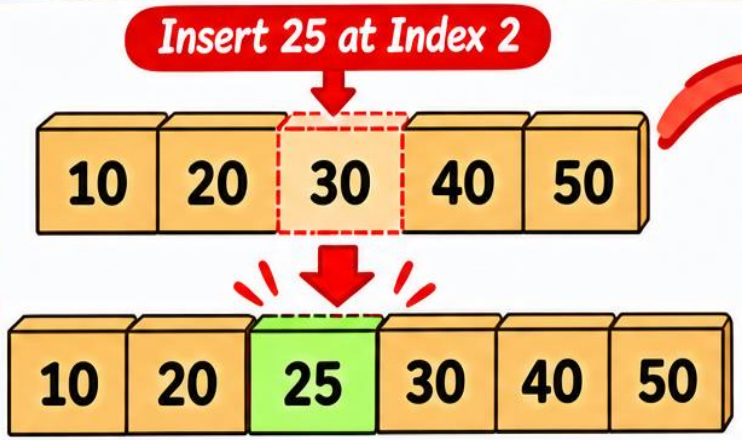


Traversal is the simplest operation. It just means "walking through" the array.
In programming, we use a simple **for loop** to do this.

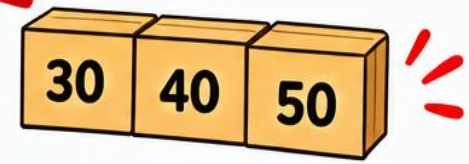
Array Operation 2 – Insertion

What is Insertion?

Adding a new element at a specific **position (index)** in the array.



Shift 30, 40, 50 to the **Right** to Make Room!



The Problem:

Arrays have a **fixed size** and are stored back-to-back.



How it works:

1. Check if the array is full **(Overflow)**.
2. Shift all elements to the right (from the insertion point to the end).
3. Place the new element in the empty spot.

Time Complexity:

O(n)

(Linear Time)

Shifting is hard work!

Mechanical Analogy:



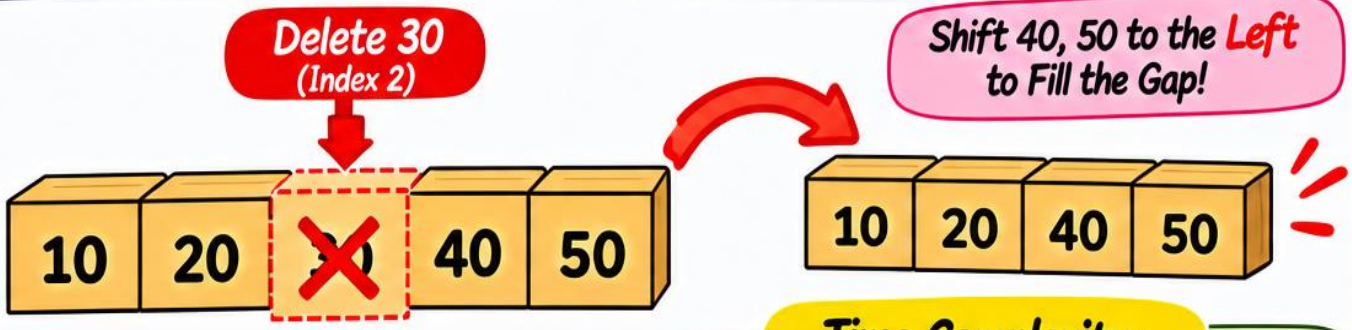
Inserting a new gear onto a tightly packed shaft. You have to slide all the gears on the right side further down the shaft to make space.

Insertion in an array is like adding a box **in the middle** of a tightly packed container. You must move the elements on the right (30, 40, 50) to make space. This shifting takes time, which is why **inserting** into a massive array can be **slow**.

Array Operation 3 – Deletion

What is Deletion?

Removing an existing element from a specific **position**.



The Problem:

You cannot leave an **empty hole** in an array.



How it works:

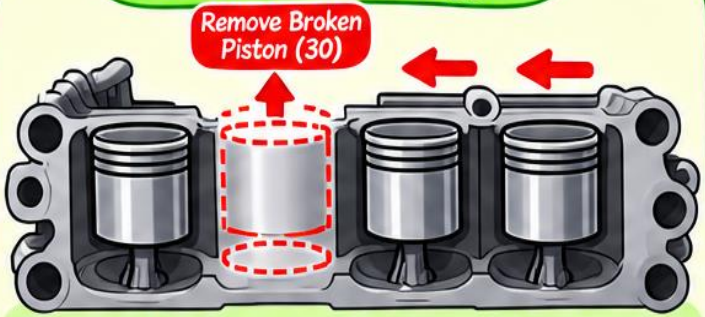
1. Check if the array is empty **(Underflow)**.
2. Find the element to delete.
3. Shift all elements to the left (from the deleted point to the end) to fill the gap.

Time Complexity:

O(n)
(Linear Time)

Because of the shifting.

Mechanical Analogy:



The remaining pistons must slide over to close the gap; you can't leave a blank space in the engine.

Deletion has the exact opposite problem of insertion. If you take out a part in the middle, you **leave a hole**. Arrays must be continuous. So, if you remove 30, the 40 and 50 must **slide to the left** to fill the space. Again, this shifting takes time, making **deletion** an **O(n)** operation.

Array Operation 4 – Searching

What is Searching?

Finding the **location (index)** of a specific element.

Method 1: Linear Search (Unsorted Array)



1. Check every element one by one from the start until found.

Time Complexity: O(n) (Slow for large data).

Method 2: Binary Search (Sorted Array ONLY)



1. Check the middle element. If the target is smaller, search the left half. If larger, search the right half.

Time Complexity: O(log n) (Extremely fast).

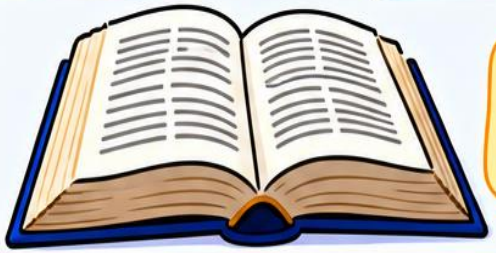
Mechanical Analogy:

Linear Search



Searching for a nut in a **random pile**.

Binary Search

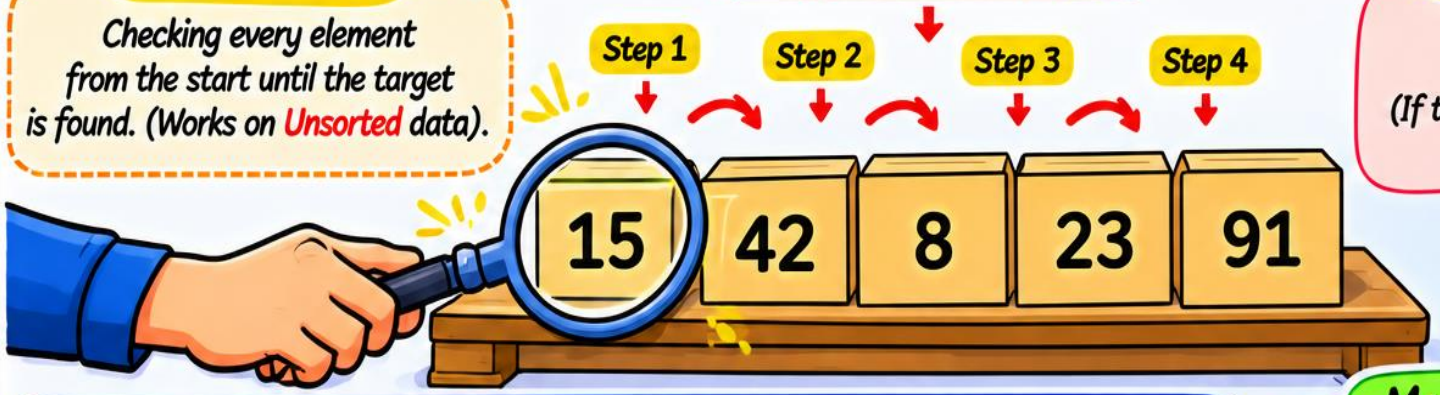


Finding a word in a **dictionary**.

Linear Search (The "One-by-One" Method)

What is it?

Checking every element from the start until the target is found. (Works on Unsorted data).



Time Complexity:

O(n)

(If there are 1000 parts, worst case you check all 1000).

Step-by-Step Example: Let's search for 23 in the array [15, 42, 8, 23, 91]

Step	Index	Value	Action / Comparison	Result
1	0	15	Is 15 == 23?	✗ No. Move to next.
2	1	42	Is 42 == 23?	✗ No. Move to next.
3	2	8	Is 8 == 23?	✗ No. Move to next.
4	3	23	Is 23 == 23?	✓ YES! Found at Index 3.

Mechanical Analogy:

It's like looking for a specific wrench in a messy, unsorted toolbox.



You pick up one tool, check if it's the right size. If not, you put it down and pick the next one.

Linear Search is simple, but slow if you have thousands of tools.

You keep doing this until you find it or the box is empty.

Binary Search (The "Divide & Conquer" Method)

What is it?

Repeatedly dividing the search interval in half. (Requires data to be **Sorted**).

Target to find: 42



Formula:

$$Mid = \frac{Low + High)}{2}$$

Time Complexity:

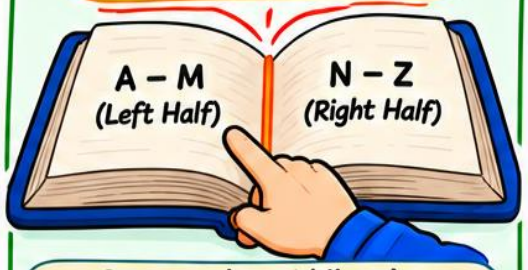
$O(\log n)$

(Extremely fast. For 1000 parts, it takes max 10 steps).

Step-by-Step Example: Let's search for **42** in the **Sorted** array [8, 15, 23, 42, 91]

Step	Low (Start)	High (End)	Mid (Middle)	Value at Mid	Comparison	Action
1	0	4	(0 + 4) / 2 = 2	23	42 > 23	Target is bigger. Ignore left half.
2	3	4	(3 + 4) / 2 = 3	42	42 == 42	✅ Found at Index 3!

Binary Search is like finding a word in a dictionary.



Open in the middle, then choose left or right half. Keep halving until you find it!

How it works?

1. Start with the whole array.
2. Find the middle element.
3. If Target > Middle → Search Right Half.
4. If Target < Middle → Search Left Half.
5. Repeat until found or the search space is empty.



Important Note for Students:

Binary search only works if the array is **Sorted**.

If the array is unsorted, you must use **Linear Search.**

Binary Search is a smart, fast,and efficient method to find data in a **sorted** array using the **Divide & Conquer** strategy!